

# Code Nexus

## Polymorphic Data Streams in the Digital Matrix

*Summary: Master abstract classes, method overriding, and subtype polymorphism while building advanced data processing pipelines that adapt and evolve in real time.*

*Version: 3.1*

# Contents


| | | |
|---|---|---|
| **I** | **Introduction** | **2** |
| **II** | **AI Instructions** | **3** |
| **III** | **General Instructions** | **5** |
| **IV** | **Exercise 0: Data Processor** | **6** |
| **V** | **Exercise 1: Polymorphic Processing of a Data Stream** | **9** |
| **VI** | **Exercise 2: Data Pipeline** | **11** |
| **VII** | **Turn in and Submission** | **13** |

# Chapter I

# Introduction

Welcome to the Code Nexus, Stream Engineer!

The year is 2087. In the sprawling digital metropolis of Neo-Tokyo, data flows through quantum fiber networks like neon-bright rivers of pure information. The **Code Nexus** stands as humanity's greatest achievement: a cybernetic cathedral where billions of data streams converge, transform, and evolve in perfect harmony.

But the Nexus does not just consume data; it **understands** it. Each data stream carries its own digital signature, its own behavioral patterns. How does a single system comprehend such diversity?

Through the principles of **abstract classes**, **polymorphism**, **method overriding**, and **inheritance hierarchies**, you will handle different types of data in the same software workflow.

*Welcome to the future. Welcome to the Code Nexus.*

# Chapter II

# AI Instructions

## Context

During your learning journey, AI can assist with many different tasks. Take the time to explore the various capabilities of AI tools and how they can support your work. However, always approach them with caution and critically assess the results. Whether it's code, documentation, ideas, or technical explanations, you can never be completely sure that your question was well-formed or that the generated content is accurate. Your peers are a valuable resource to help you avoid mistakes and blind spots.

## Main message

- 👉 Use AI to reduce repetitive or tedious tasks.
- 👉 Develop prompting skills — both coding and non-coding — that will benefit your future career.
- 👉 Learn how AI systems work to better anticipate and avoid common risks, biases, and ethical issues.
- 👉 Continue building both technical and power skills by working with your peers.
- 👉 Only use AI-generated content that you fully understand and can take responsibility for.

## Learner rules:

- You should take the time to explore AI tools and understand how they work, so you can use them ethically and reduce potential biases.
- You should reflect on your problem before prompting — this helps you write clearer, more detailed, and more relevant prompts using accurate vocabulary.
- You should develop the habit of systematically checking, reviewing, questioning, and testing anything generated by AI.
- You should always seek peer review — don't rely solely on your own validation.

## Phase outcomes:

- Develop both general-purpose and domain-specific prompting skills.
- Boost your productivity with effective use of AI tools.
- Continue strengthening computational thinking, problem-solving, adaptability, and collaboration.

## Comments and examples:

- You'll regularly encounter situations — exams, evaluations, and more — where you must demonstrate real understanding. Be prepared, keep building both your technical and interpersonal skills.
- Explaining your reasoning and debating with peers often reveals gaps in your understanding. Make peer learning a priority.
- AI tools often lack your specific context and tend to provide generic responses. Your peers, who share your environment, can offer more relevant and accurate insights.
- Where AI tends to generate the most likely answer, your peers can provide alternative perspectives and valuable nuance. Rely on them as a quality checkpoint.

**✓ Good practice:**

> I ask AI: "How do I test a sorting function?" It gives me a few ideas. I try them out and review the results with a peer. We refine the approach together.

**✗ Bad practice:**

> I ask AI to write a whole function, copy-paste it into my project. During peer-evaluation, I can't explain what it does or why. I lose credibility — and I fail my project.

**✓ Good practice:**

> I use AI to help design a parser. Then I walk through the logic with a peer. We catch two bugs and rewrite it together — better, cleaner, and fully understood.

**✗ Bad practice:**

> I let Copilot generate my code for a key part of my project. It compiles, but I can't explain how it handles pipes. During the evaluation, I fail to justify and I fail my project.

# Chapter III

# General Instructions

- Your project must be written in **Python 3.10 or later**.
- Your project must adhere to the **flake8** coding standard.
- All code must include comprehensive type annotations. Check this using `mypy` .
- Exception handling should protect the data streams from corruption.
- Authorized imports: `abc` and `typing` .
- All standard classes and collections are authorized, along with their methods (`int`, `str`, `list`, `dict`, etc.).
- All built-in functions are authorized.

# Chapter IV

# Exercise 0: Data Processor

|  | Exercise0 |
|---|---|
| data_processor | |
| Directory: *ex*0/ | |
| Files to Submit: `data_processor.py` | |
| Authorized: `builtins, standard types, import typing, import abc` | |

**Engineering Briefing**: Welcome to the Code Nexus! Build the foundation of our data processing system. You will create the base processor architecture and demonstrate how different data types can share common processing interfaces while maintaining their unique characteristics.

This exercise requires the use of *abstract classes* using `ABC` (Abstract Base Class). We will first create separate classes that share common interfaces. In the next exercise, they will be unified in the same workflow.

Set up the following architecture:

- An abstract class `DataProcessor` that inherits from `ABC` and defines the common processing interface.
- Three specialized classes `NumericProcessor`, `TextProcessor`, and `LogProcessor` that inherit from the `DataProcessor` class and will process different kinds of data.
- Two abstract methods in `DataProcessor`: `validate`, which will check whether the input data are appropriate for the current data processor, and `ingest`, which will process the input data. Each specialized class will need to override these methods.
- One standard method in `DataProcessor`: `output`, which will output ingested data.

You need to comply with the following constraints:

- The `validate` method will be defined as `validate(self, data: Any) -> bool` in the `DataProcessor` class. The overriding methods in the specialized classes will share the same signature, as they cannot know what data will be sent and must

accept any type. This method returns a bool that indicates if the provided data can be ingested by this data processor.

- The `ingest` method will be defined as `ingest(self, data: Any) -> None` in the `DataProcessor` class. The overriding methods in the specialized classes will have their own specific signatures to match the types they expect. In case the user does not `validate` the data before calling `ingest`, and provides invalid data, an exception must be raised.
- The `output` method will be defined as `output(self) -> tuple[int, str]` in the `DataProcessor` class. There is no need to override it in the specialized classes.
- The `NumericProcessor` ingests `int`, `float`, and lists of both types (including mixed-type lists). It then converts the data into strings and stores it internally (keeping each item separated), waiting to be extracted piece by piece using the `output` method. The overriding `ingest` method signature must reflect the accepted types.
- The `TextProcessor` ingests `str` and lists of strings. It stores the data internally (keeping each item separated), waiting to be extracted piece by piece using the `output` method. The overriding `ingest` method signature must reflect the accepted types.
- The `LogProcessor` ingests a `dict` of string key-value pairs, and lists of that type. It then converts the data into strings and stores it internally (keeping each item separated), waiting to be extracted piece by piece using the `output` method. The overriding `ingest` method signature must reflect the accepted types.
- The `output` method will extract the oldest piece of data stored internally in the data processor, along with the associated processing rank within the data processor. The piece of data is then removed from the data processor.

Finally, test your architecture:

- Create instances for each specialized class.
- Test valid and invalid data for each class through the `validate` method.
- Test at least one invalid data item with the `ingest` method without prior validation, and check that it raises an exception. This will leave you with a `mypy` warning, on purpose.
- Ingest various data for each data processor and then extract it using `output`.

Example:

```
$> python3 data_processor.py
=== Code Nexus - Data Processor ===

Testing Numeric Processor...
 Trying to validate input '42': True
 Trying to validate input 'Hello': False
 Test invalid ingestion of string 'foo' without prior validation:
 Got exception: Improper numeric data
 Processing data: [1, 2, 3, 4, 5]
 Extracting 3 values...
 Numeric value 0: 1
 Numeric value 1: 2
 Numeric value 2: 3

Testing Text Processor...
 Trying to validate input '42': False
 Processing data: ['Hello', 'Nexus', 'World']
 Extracting 1 value...
 Text value 0: Hello

Testing Log Processor...
 Trying to validate input 'Hello': False
 Processing data: [{'log_level': 'NOTICE', 'log_message': 'Connection to server'}, {'log_level': 'ERROR
     ', 'log_message': 'Unauthorized access!!'}]
 Extracting 2 values...
 Log entry 0: NOTICE: Connection to server
 Log entry 1: ERROR: Unauthorized access!!
```

# Chapter V

# Exercise 1: Polymorphic Processing of a Data Stream

|  | Exercise1 |
|---|---|
| data_stream | |
| Directory: *ex*1/ | |
| Files to Submit: `data_stream.py` | |
| Authorized: `builtins, standard types, import typing, import abc` | |

**Engineering Briefing:** It is time to build an adaptive stream processing workflow that can handle multiple data types simultaneously.

Use your code from Exercise 0 and improve it:

- Create a `DataStream` class that will receive a stream of data containing different types and then will route each element to the appropriate data processor using polymorphic behavior.
- This class will implement the
`def register_processor(self, proc: DataProcessor) -> None:`
method that allows you to register a new data processor to process the data stream.
- This class will implement the
`def process_stream(self, stream: list[typing.Any]) -> None:`
method that will analyze each element of the list received as a parameter and send it to the appropriate registered data processor. Error messages will be printed if no data processor can handle an element.
- Finally, the class will implement the `def print_processors_stats(self) -> None:` method in order to print stream statistics.
- Create a test scenario that demonstrates the correct processing of a data stream. Display statistics on registered data processors, consume elements using the `output` method of each data processor and show updated statistics.

Example:

```
$> python3 data_stream.py
=== Code Nexus - Data Stream ===

Initialize Data Stream...
== DataStream statistics ==
No processor found, no data

Registering Numeric Processor

Send first batch of data on stream: ['Hello world', [3.14, -1, 2.71], [{'log_level': 'WARNING', '
    log_message': 'Telnet access! Use ssh instead'}, {'log_level': 'INFO', 'log_message': 'User wil is
    connected'}], 42, ['Hi', 'five']]
DataStream error - Can't process element in stream: Hello world
DataStream error - Can't process element in stream: [{'log_level': 'WARNING', 'log_message': 'Telnet
    access! Use ssh instead'}, {'log_level': 'INFO', 'log_message': 'User wil is connected'}]
DataStream error - Can't process element in stream: ['Hi', 'five']
== DataStream statistics ==
Numeric Processor: total 4 items processed, remaining 4 on processor

Registering other data processors
Send the same batch again
== DataStream statistics ==
Numeric Processor: total 8 items processed, remaining 8 on processor
Text Processor: total 3 items processed, remaining 3 on processor
Log Processor: total 2 items processed, remaining 2 on processor

Consume some elements from the data processors: Numeric 3, Text 2, Log 1
== DataStream statistics ==
Numeric Processor: total 8 items processed, remaining 5 on processor
Text Processor: total 3 items processed, remaining 1 on processor
Log Processor: total 2 items processed, remaining 1 on processor
```



How does polymorphism allow the DataStream to handle different data types in the stream without knowing their specific implementations? What are the benefits of this design approach?

# Chapter VI

# Exercise 2: Data Pipeline

|  | Exercise2 |
|---|---|
| data_pipeline | |
| Directory: *ex2/* | |
| Files to Submit: `data_pipeline.py` | |
| Authorized: `builtins, standard types, import typing, import abc` | |

**Engineering Briefing:** Your final challenge is to integrate everything into a complete data processing pipeline that demonstrates mastery of polymorphic architecture at an enterprise scale.

Use your code from Exercise 1 and improve it in order to obtain a complete data pipeline. Your `DataStream` class already handles input streams correctly. You need now to handle the output part of the pipeline. This will be achieved by using a plugin system for export classes, made export-compatible through *duck typing*.

Implement the following:

- A new `ExportPlugin` class that inherits from the special `Protocol` class.
- This class will define the following method, which will act as a constraint for each export plugin:
  `def process_output(self, data: list[tuple[int, str]]) -> None:`
  The type of the `data` parameter is a list of tuples that matches the return value of the `output` method from the `DataProcessor` class.
- The `DataStream` class will now implement the
  `def output_pipeline(self, nb: int, plugin: ExportPlugin) -> None:`
  method, to be used after calling `process_stream`, that will consume `nb` elements from all registered data processors and export them using the provided compatible plugin.
- Create at least a CSV export plugin and a JSON export plugin. No need to use a specific import for these plugins, manually create valid CSV and JSON strings.

Example:

```
  $> python3 data_pipeline.py
=== Code Nexus - Data Pipeline ===

Initialize Data Stream...

== DataStream statistics ==
No processor found, no data

Registering Processors

Send first batch of data on stream: ['Hello world', [3.14, -1, 2.71], [{'log_level': 'WARNING', '
    log_message': 'Telnet access! Use ssh instead'}, {'log_level': 'INFO', 'log_message': 'User wil is
    connected'}], 42, ['Hi', 'five']]

== DataStream statistics ==
Numeric Processor: total 4 items processed, remaining 4 on processor
Text Processor: total 3 items processed, remaining 3 on processor
Log Processor: total 2 items processed, remaining 2 on processor

Send 3 processed data from each processor to a CSV plugin:
CSV Output:
3.14,-1,2.71
CSV Output:
Hello world,Hi,five
CSV Output:
WARNING: Telnet access! Use ssh instead,INFO: User wil is connected

== DataStream statistics ==
Numeric Processor: total 4 items processed, remaining 1 on processor
Text Processor: total 3 items processed, remaining 0 on processor
Log Processor: total 2 items processed, remaining 0 on processor

Send another batch of data: [21, ['I love AI', 'LLMs are wonderful', 'Stay healthy'], [{'log_level': '
    ERROR', 'log_message': '500 server crash'}, {'log_level': 'NOTICE', 'log_message': 'Certificate
    expires in 10 days'}], [32, 42, 64, 84, 128, 168], 'World hello']

== DataStream statistics ==
Numeric Processor: total 11 items processed, remaining 8 on processor
Text Processor: total 7 items processed, remaining 4 on processor
Log Processor: total 4 items processed, remaining 2 on processor

Send 5 processed data from each processor to a JSON plugin:
JSON Output:
{"item_3": "42", "item_4": "21", "item_5": "32", "item_6": "42", "item_7": "64"}
JSON Output:
{"item_3": "I love AI", "item_4": "LLMs are wonderful", "item_5": "Stay healthy", "item_6": "World hello
    "}
JSON Output:
{"item_2": "ERROR: 500 server crash", "item_3": "NOTICE: Certificate expires in 10 days"}

== DataStream statistics ==
Numeric Processor: total 11 items processed, remaining 3 on processor
Text Processor: total 7 items processed, remaining 0 on processor
Log Processor: total 4 items processed, remaining 0 on processor
```

# Chapter VII

# Turn in and Submission

Turn in your assignment to your `Git` repository as usual. Only the work inside your repository will be evaluated during the defense. Do not hesitate to double-check the names of your files to ensure they are correct.



During evaluation, you may be asked to explain polymorphic behavior, demonstrate method overriding, extend your systems with new data types, or modify processing behavior. Make sure you understand how inheritance enables code reuse while allowing behavioral specialization.



You only need to submit the files requested by the subject of this project.